

1.5 Basic Principles of Large Language Models

To use AI tools in coding, you don't need to understand how they work. In this respect, you can simply skip this section. Nevertheless, it's useful if you at least know the most important terms relating to *large language models* (LLMs).

More Depth

Don't worry, this chapter isn't a comprehensive AI basics course! There are already enough books available on this subject. We can particularly recommend *Developing AI Applications: An Introduction* by Metin Karatas, also published by Rheinwerk Computing.

If you want to get to know and understand AI and LLM basics with more technical details, we also recommend Mario Zechner's basic course. The course consists of multiple Jupyter notebooks that invite you to experiment: <https://github.com/badlogic/genai-workshop>.

1.5.1 From the Human Brain to Neural Networks

Let's get started with the human brain, which consists of around 85 billion, that is, approximately 10^{11} , nerve cells (neurons). Each neuron is connected to many other neurons by synapses. In total, there are approximately 100 trillion = 10^{14} such connections. During learning, individual synapses are strengthened or weakened. So, when you memorize something or when you learn something, the synapses of your brain (i.e., the "wiring") change.

Compared to a computer, the human brain is amazingly good at processing information in parallel: seeing, hearing, thinking, and speaking. The brain also uses energy extremely efficiently and is content with around 20 watts.

That is about one-fifth of the energy expenditure of an adult and significantly less than is needed to run LLMs on a computer.

For around 80 years, attempts have been made to recreate neurons and synapses as much as possible using artificial neural networks. From a mathematical perspective, a neural network receives and processes digital data (input), and then returns the result (output). Input and output are represented as vectors, that is, as sets of numbers. The content of the vectors depends on the application: text, audio data, the pixels of an image sensor, and so on.

The input vectors are processed into an output vector by matrix multiplication. This process usually involves multiple stages. The numbers (“weights”) contained in the matrixes are also referred to as the parameters of the neural network. They correspond to the synapses of the human brain.

The first neural networks failed for two reasons: the number of parameters was too small to create an adequate reflection of reality, and the computing power was too low to train the model sufficiently. Today’s language models consist of many billions of parameters. graphics processing units (GPUs) help to train the model. Increasingly sophisticated algorithms for designing the mathematical model make this concept extremely successful today.

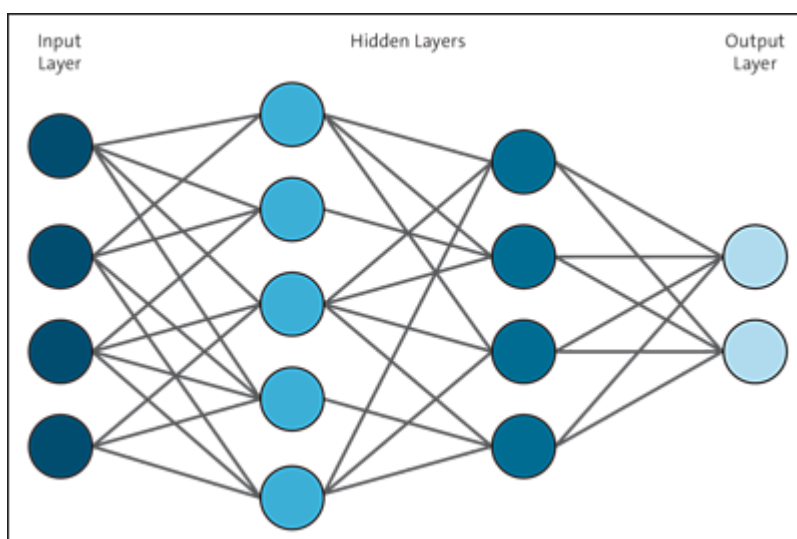


Figure 1.8 Diagram of a Neural Network

1.5.2 Machine Learning through Backpropagation

Self-learning algorithms for machine learning were an important milestone in the development of neural networks. One learning concept is to train the model with large amounts of familiar learning material, such as images of plants. With each training image, the contents of the matrixes are changed via *backpropagation* so that the deviations between the known results of the training material and the results calculated by the neural network are minimized. The program is then able to recognize the plants depicted in new photos (which were not part of the training) with a high probability.

Training can be implemented in an even smarter way by using systems that are clearly defined by rules: A computer program can play chess against itself and optimize the matrices of its neural model so that it wins as many games as possible. Today, computer programs based on the AlphaZero algorithm play chess and Go better than any human—and that without access to opening libraries and without practice material from world champion games.

Current AI systems are optimized for a specific purpose. They can't be used universally for every conceivable task. The optimization concerns both the structure and linking of the neural network as well as the learning algorithm.

Compared to the human brain, neural networks are still “simple.” There are many more connections with different neurotransmitters in the human brain. The overall structure is therefore finer meshed and more varied than the artificial replica. In addition, the human brain can reorganize itself to a limited extent.

For all of these reasons, the human brain is superior to its artificial counterparts in its universal orientation. On the other hand, a computer (with or without neural networks) can perform special tasks faster.

1.5.3 Large Language Models

LLMs are a special case of neural networks that are specifically optimized for dealing with language. LLMs “understand” text instructions and can answer questions. Programming languages are ultimately just languages, albeit quite specialized ones. LLMs that have been trained with a sufficient number of code examples can therefore program surprisingly well. All the AI tools described in this book are based on LLMs.

Children begin to learn a language by hearing parents and other people speak, articulating their first words often still quite indistinctly. In nursery school, they sing songs with other children. They learn to read and write at elementary school. Over the course of nearly 20 years, the child acquires more and more words and the underlying grammar. They constantly receive feedback, interact with their environment, and learn how to communicate as well as possible. The teenager can now communicate by speaking and writing, understands complex texts, and can formulate them themselves. If you’re reading this book, you’ve already largely completed this learning process. Nevertheless, you’re still expanding your vocabulary with new technical terms!

LLMs work in a very similar way. During training, which only takes a few months in enormous server farms, the neural network is fed with gigantic amounts of text—the entire Wikipedia, the code collected on GitHub, entire libraries of books, Facebook postings, and so on. (The extent to which this training is permissible at all without the consent of the authors of these texts must first be clarified in court proceedings.)

What Training Material Was Used?

Presumably because of the legal uncertainty just mentioned, AI companies are revealing less and less about which data they have used for training. The sources of the free language model Llama 1 are broken down relatively precisely. The training material consists of approximately 1,200 billion tokens (<https://huggingface.co/datasets/togethercomputer/RedPajama-Data-1T>). This corresponds to around 2.4 billion pages of text in the format of this book, or almost 7 million books with roughly the same amount of content as this book.

Almost 90% of the training material comes from various publicly accessible websites, around 5% from GitHub and around 2% from Wikipedia. The rest of the materials are publicly accessible scientific publications (arXiv) and Stack Exchange contributions.

Various training data sets are available to download from the internet for AI research: <https://huggingface.co/datasets>.

Current LLMs store this knowledge in models comprising many billions of parameters. In principle, these parameters correspond to the weighting of connections between the nodes of the neural network. (In the human brain, too, the synapses between the neurons are designed differently; that is, they transmit a signal more or less strongly. During learning, the synapses change in the brain, and the weightings change in the neural model.)

LLMs differ not only in the number of parameters but also in the accuracy (*quantization*) with which they are stored. Often only 4 bits are used for this. This means that only 16 different values can be saved, but two parameters can be stored in 1 byte to save space.

For example, the freely available language model `llama3:8b` comprises 8 billion parameters (8b in `llama3:8b` stands for 8 billion). The total memory requirement of this model is around 5 GB because in addition to the parameters, some other data is stored.

There are currently attempts to save parameters with only 1 bit or with 1.58 bits (b1.58). The odd number results when *three* states are saved for each parameter: 0, 1, and -1. This kind of economical quantification allows for models with even more parameters with the same amount of memory. However, it still needs to be proved whether this is really advantageous. The efficient use of these so-called 1-bit LLMs also requires new hardware.

Bigger Is Better or Small Is Beautiful?

Language models with more parameters understand more complex questions and usually give better answers. However, they also require more resources, both during training and later in their application. The language model of ChatGPT-4, which isn't publicly accessible, probably comprises

more than 1,000 billion parameters. The largest variant of the free Llama language model has 400 billion parameters (as of mid-2024).

However, the number of parameters isn't the only criteria. The quality of language models is heavily dependent on the material used during training and in fine-tuning. Finally, language models can be focused on content. Some LLMs are explicitly intended for programming. For this reason, a particularly large number of code examples were used during the training.

One of the most exciting AI research topics at the moment is the attempt to make high-quality language models as small as possible. *Small language models* (SLMs) would allow local application on notebooks and even smartphones. "Small" is relative; such models still comprise several billion parameters.

One approach isn't to use terabytes of undifferentiated texts collected from the internet for training, but to select less, higher-quality training material. It remains to be seen whether and when the quality of such "small" models can keep up with large commercial models.

1.5.4 The Transformer Model

A special feature of LLMs is the underlying transformer model. The model is based on a research breakthrough published in 2017 under the title "Attention Is All You Need" (available free at <https://arxiv.org/abs/1706.03762>). This relatively new AI technique applies neural networks to sequences, for example, to sentences made up of words. In the case of LLMs, however, *tokens* rather than words are used as data units: these are parts of a word, comparable to partial words or syllables.

Tokens are mapped internally by vectors. A vector is simply a sequence of numbers (an array). These vectors are relatively large and consist of 768 or 2,048 values, for example. Each value is mapped as a 16-bit or 32-bit floating point number. The space requirement of a token is therefore several kilobytes. The same word would only take up a few bytes in UTF-8 encoding.

The lavish use of internal representation has the advantage that a token doesn't simply store the characters of the word, but countless pieces of contextual information. Words that are related in content have similar vectors. The *tokenizer*, which is upstream of the transformer, is responsible for converting words into token vectors.

In general, a transformer turns a data sequence into a new data sequence. An easy-to-understand application is the translation of a text from one language into another. An encoder converts the input into an internal display. After processing, the decoder takes care of turning this data back into text.

The transformer model is characterized by the following features:

- The self-attention mechanism evaluates a word or its token in the context of one or more sentences.
- The transformer models process sequences in several successive layers (self-attention, feed-forward, etc.). This multilayer approach helps to recognize complex relationships.
- The transformer model dispenses with feedback (called *recurrent layers*). Compared to other AI models for language comprehension, this is more efficient. However, additional information about the position of words within a sentence must now be stored using *positional encoding* so that the logic of a sentence or that of computer code isn't lost.

During training, the transformer compresses the knowledge collected in huge amounts of text into comparatively small parameter sets. For example, 100 TB of text (10^{14} bytes) can be processed into a language model with 20 billion parameters. Its space requirement is 10^{10} bytes = 10 GB with a 4-bit accuracy per parameter.

Transformer Alternatives

The transformer model has helped LLMs achieve a breakthrough. It works amazingly well, but there are disadvantages. In particular, the resources required for training and application are very high.

AI research is therefore endeavoring to improve the transformer model or to take completely different paths to achieve equivalent results with less

computing effort. It's possible that the transformer model will be a thing of the past in a few years' time, and new approaches will prevail.

If you're interested in AI research and are ready to dive into technical details, we have two reading tips for you:

- “xLSTM: Extended Long Short-Term Memory”
(<https://arxiv.org/abs/2405.04517>)
- “Scalable MatMul-free Language Modeling”
(<https://arxiv.org/abs/2406.02528>)

1.5.5 Generative Pretrained Transformer (GPT)

Chat Generative Pretrained Transformer (ChatGPT) and comparable programs can communicate in text form and are chatbots. The special feature compared to conventional, less sophisticated chatbots is that a previously trained transformer language model is used.

The central term *generative* in ChatGPT refers to generating new text to answer a question (a “prompt”). GPTs differ in various details from the original transformer model, which consists of an encoder and a decoder. The most important innovation is that GPTs focus on the decoder aspect: the decoder-only architecture doesn't convert existing text into another form as before, but creates new text to answer the given prompt.

An autoregressive model is used to generate the response: once a sentence has been started, it decides which is the most likely or most appropriate continuation in the given context. In this way, the answer is produced word for word without there being an internal representation of the entire answer beforehand.

Due to a certain randomness in the decision-making process (e.g., to choose between two similarly matching sequels), each answer is individual. This is why GPTs always provide varying answers to the same question. If you're not satisfied with the first answer, you can regenerate it with most programs and, with a bit of luck, get a better result.

Temperature

With some AI tools, you can set the “temperature.” This option is available for almost all language models that you use via an API. The temperature determines how random the answer is. When AI tools are used for coding, too much randomness, that is, a high temperature, is rarely a good idea. Just think of the result if you’re sick and feverish and try to finish programming something quickly.

On the other hand, there are cases where a little more creativity is required. Then, it may be worth experimenting with the AI temperature.

Even though the abbreviation GPT was coined as part of the ChatGPT brand name, GPT is now considered a generic term for LLMs that work in a similar way to ChatGPT.

1.5.6 Training: From Training Material to Language Model

The autoregressive model of GPTs works so well because the text prediction was previously practiced with millions of pages of different text. During training, the language model tries to guess the next word. The model is optimized step-by-step on the basis of deviations from existing training texts. Conveniently, this training can be carried out almost fully automatically.

Many billions of matrix multiplications must be performed during training. This is currently achieved most efficiently with GPUs or GPU-like processors. The largest manufacturer of such GPUs is NVIDIA. This is the reason for the huge current market value of this company.

In the course of the training, texts amounting to several terabytes are processed (1 terabyte of text corresponds to about 500 million book pages, which is about 1.7 million different books with a similar volume to this book). Huge server farms with GPUs are used for training.

Optimization of Training Material, Fine-Tuning, and Censoring

Further processing steps often take place before, during, or after training, which we won't discuss in detail here:

- During *data set curation*, unsuitable content (e.g., pornographic or racist texts, depending on the objective) is removed from the training material.
- The model is optimized during *fine-tuning*. For example, *biases* contained in the training material are corrected.
- *Censoring* attempts to prohibit the model from answering certain questions (e.g., “How do I produce a nerve gas?”).

The result of the training is the finished language model, that is, the compact storage of the parameters that have just been calculated. The file of a language model is significantly smaller than the training material.

Depending on the number of parameters, the size ranges from a few gigabytes for models intended for the local execution to several hundred gigabytes for the very large commercial models from OpenAI and others.

We often read that training is very energy-intensive and that huge amounts of training material are processed during training. Could this perhaps be a little more precise? To avoid being seen as energy wasters, AI companies are now more reticent about providing detailed information on the training process.

We've found relatively precise data on the free Llama 3.1 language model. In July 2024, Meta completed the training of the model, which is available in three sizes (8, 70, and 400 billion parameters). The training material consisted of approximately 60 TB of text. The training of all three variants required a total of almost 40 million GPU hours with an output of 700 W per GPU. This results in an energy requirement of just under 28 GWh. You can read more details about the model here:

https://github.com/meta-llama/llama-models/blob/main/models/llama3_1/MODEL_CARD.md

It's hard to imagine how much 28 GWh of electricity is, so here's a comparison: Private electricity consumption per household in Germany amounted to an average of 3,400 kWh per year in 2021. Llama 3.1's training therefore consumed about as much electricity as 8,200 private households in an entire year.

1.5.7 Large Language Model Application: From Prompt to Response

Once a precalculated language model is in place, it's still a long way from sending the question (the prompt) to displaying the answer (the response). If you use ChatGPT, the result appears virtually in real time, or at least faster than you can read it. The high speed belies the number of processing steps required:

- **Tokenization/encoding**

First, your input is converted into tokens that the language model can understand.

- **Token generation**

An algorithm produces a sequence of output tokens based on the language model and the input tokens. Basically, each additional token is calculated as the most probable or plausible continuation of the previous output tokens.

- **Decoding**

The token stream is then converted back into language that humans can understand.

- **Postprocessing**

Various downstream processes ensure that the sentences are grammatically correct, that the punctuation is correct, and so on. The answer is also presented in an appealing layout (headings, highlighting, separate font for code, etc.).

These steps require billions of arithmetic operations. The effort becomes noticeable as soon as you try to run a language model locally on your computer (see [Chapter 9](#)). For slightly larger models with more than 10 billion parameters, even powerful notebooks with a lot of RAM are overwhelmed. The answers appear agonizingly slowly on the screen while the CPU or GPU are running hot. You're then faced with the following dilemma:

- You use an SLM. It's quite possible that the future belongs to task-specific language models. (A language model focused on coding doesn't necessarily have to shine with biological, geographical, or mathematical knowledge.)
- You purchase expensive hardware, for example, a desktop computer with a powerful graphics card. This significantly increases the speed, often measured in tokens per second. However, the model size is still limited by the memory (video RAM [VRAM]) accessible to the GPU.
- You stick to cloud-based solutions, which may be subject to a charge (as with GitHub Copilot).

LLM-Compatible Hardware

In the coming years, we expect notebooks to become LLM-capable, that is, capable of running language models locally with good quality and speed. This requires two things: sufficient RAM and hardware that can perform many matrix multiplications in a short time. The way there is either via more powerful GPUs (graphics processors) or via special *neural processing units* (NPUs).

NPUs are already integrated into modern CPUs, which promise a computing power of several teraflops per second (1,000 billion floating point operations per second). However, as of early 2025, it's fairly unclear what NPUs will actually be used for. Marketing is certainly the most important point. In second place is cropping out the person in front of the notebook camera in video conferences.

However, there is a lack of software drivers and established standards for the use of NPUs in the execution of local language models. In addition, the computing power of NPUs is currently significantly lower than that of GPUs. In this respect, it remains to be seen whether NPUs can really accelerate the execution of local language models.

1.5.8 The Limits of Linguistic “Intelligence”

Current language models can explain recursion in the language of Shakespeare. The result is fantastic!

Prompt: Explain recursion to me in poem form. Try to imitate the language of Shakespeare.

Recursion, harken! A most curious art,
Wherein the selfsame path doth oft restart.
A function, bold, doth to itself return,
And through this loop, the mind begins to yearn.

[Three more stanzas follow, all in a similar quality.]

As amazing as the linguistic qualities are, you must never forget that you're dealing with a language model. Such models are subject to major restrictions in many respects. There is absolutely no indication of a "general intelligence," as evidenced here:

- Language models can't calculate.
- They can't draw (e.g., create diagrams).
- They only understand text, so they can't handle image, audio, and video data.
- They fail to reach logical conclusions that are relatively simple for humans.
- Their knowledge ends with the cut-off time for training. Language models can't access up-to-date information from the internet.

You may now object that these restrictions do not (all) apply. In fact, it looks as if modern AI chatbots are quite capable of calculating, evaluating current websites, following voice instructions, or describing images. However, all of this is only possible if language models are extended with external additional functions using various tricks. A few typical procedures are summarized here:

- Although language models can't draw, they can provide code for common diagram description languages (PlantUML, Mermaid, etc.).
- Language models can be linked to *agents*. For example, a calculation agent analyzes the text and forwards the figures it contains to a calculation program. The language model packs the result back into an answer in text form. Agents can also evaluate files attached to a prompt, try out code and search for errors contained therein, and so on. Agents are

therefore an established tool for connecting a language model with external functions.

- Using RAG, as mentioned earlier, a language model can be linked to a constantly updated knowledge database. We describe this procedure in more detail in [Chapter 12](#).
- Multimodal models move away from pure text processing and expand the communication basis to include sound and images. Special AI tools (*not* language models!) can even edit images and videos.

All of this takes us further and further away from the central topic of this book—the application of language models to code. As long as it’s only about code, language models work amazingly well. Programming languages have a much clearer, more unambiguous syntax compared to “real” languages. This helps with the training of language models.

However, you should always be aware of the conceptual limitations of language models. Language models aren’t really intelligent. Instead, they imitate human speech behavior on the basis of their training or generate code that is structured similarly to code from the training material.

Imitation Intelligence

The software developer Simon Willison, who works intensively with AI and language models in particular, uses the term *imitation intelligence* to deny language models genuine intelligence. (If you have three quarters of an hour to spare, you should listen to the Willison’s presentation, “Imitation Intelligence” at <https://simonwillison.net/2024/Jul/14/pycon>.) However, just because a tool is flawed, that doesn’t mean it’s not useful. For comparison here, let’s consider spellcheck. This often works well, but anyone who writes text from time to time knows the control function is far from perfect. The application of language models to coding is similar: if you understand the limitations and expect occasional errors, then there is still one tool left that you shouldn’t do without.

1.5.9 The Size of the Context Window

The generation of code by AI tools in particular sounds scary and implausible. How can statistical procedures decide the content of the next line? The

process works for several reasons:

- The language model was trained with millions of lines of code. It therefore knows common programming languages, common functions and algorithms, and so on.
- The syntax of programming languages is much simpler than that of “real” languages. In English, you can express one and the same thing in an almost infinite number of ways, linguistically more or less elegant, in short sentences or in convoluted constructions with foreign words, irony, cynicism, and so on. In this respect, coding for language models is the simple special case.
- Finally, language models not only take into account the actual question but also the connection with previous questions and answers as well as with the code already available. All of this data forms the context for the continuation of the answer. Accordingly, the clearer you formulate your question, the more meaningful variable and function names your existing code uses, and so on, the better results you’ll get.



If you’re editing a file of a large software project, it would actually be ideal if the language model had the content of *all* code files available as context information. The language model would then know all classes with their methods and properties, the structure of the database schema, CSS attributes, and so on. Newly generated code would fit perfectly with the existing project.

Unfortunately, this isn’t working at the moment. The size of the context memory (often called *context window* instead) is very limited. While the language model itself is many gigabytes in size, the context memory has room for just a few thousand tokens. The function of the context memory becomes clearer with an analogy to a computer: Even if you have a huge, terabyte-sized database on your computer’s SSD, you’re limited by the size of your computer’s RAM when processing data.

Here’s the obvious question, of course: Why doesn’t the context memory simply get enlarged? In fact, this has already been done in recent months. GPT-4o supports a context memory of up to 32,000 tokens, which corresponds to around 60 pages of text. That is quite a lot. Other language models can handle even more context (200,000 tokens for Claude, 100 million

tokens for Magic AI's experimental language models). But now, new problems are arising:

- The larger the context data, the longer it takes to respond to prompts. This effect is particularly dramatic in the local execution of LLMs.
- As the context size increases, the quality of the answer decreases instead of increasing. The LLM tends to be confused by all the additional information. It's not possible to separate essential from nonessential information (*increased noise*). The self-attention mechanism, which is an essential component of the GPT algorithms, also copes poorly with too much context.
- Another limit, the maximum length of the answer, comes to the fore. The *output token limit* is often much more restrictive than the context window limit and is only a few thousand tokens for many language models.

Of course, AI research is working on improving the context issues outlined here. But until that happens (if it happens at all), AI tools work best when the question is manageable and the amount of code to be processed is no longer than around 200 to 300 lines (as of early 2025). In this book, we've repeatedly encountered fundamental limitations of AI models, and they almost always had to do with a context window that was too small for the application (see also [Chapter 10](#)).

1.5.10 The Fill-in-the-Middle Approach

If you use language models via chat, the model simply responds to your questions (prompts). It can take into account the code passed to the prompt as well as the previous questions and answers of the session.

However, wizards such as GitHub Copilot must suggest usable code *without* a specific question. This can only work satisfactorily if the AI generator takes into account the code above *and* below the current cursor position as context.

This *fill-in-the-middle* (FIM) approach contradicts the purely forward-looking GPT model. It's not enough to find the most plausible continuation for a partially completed answer if the existing code underneath is ignored.

For FIM to work well, this use case must be taken into account as early as during the training of the language model. When creating the language model, then not only is the continuation of text trained but also the insertion of text between existing text or code segments.

LLMs such as GPT-4o from OpenAI were trained simultaneously for *both* use cases, that is, both for chat-based operation (purely forward-oriented) and for use as a code or text wizard (FIM). In the case of smaller, freely available models for local use, on the other hand, there are often different variants of the model that have been explicitly optimized for specific use cases. Accordingly, with GitHub Copilot alternatives such as Continue, you can specify *two* local models, one for chat tasks and a second for the code wizard that uses the FIM method.